MASTER IN ARTIFICIAL INTELLIGENCE

Introduction to Machine Learning

CLASSIFICATION WITH LAZY LEARNING, SUPPORT VECTOR MACHINES AND RANDOM FORESTS

Authors

Soham Chakraborty

Luis David Landicho Antonio

Bhavan Mullur Gopal

Vinay Narayanappa

October 2026

1 Introduction and Objectives

In machine learning, classification is a task in which a model assigns observations to predefined categories based on their input features. There are different classification algorithms that approach this task in various ways. These algorithms exhibit varying predictive performance and computational requirements depending on the characteristics of the dataset, the model configuration, and the amount of training data available. Therefore, evaluating alternative classification approaches under a common experimental framework is important for understanding their strengths, limitations, and practical trade-offs.

This study investigates three classification algorithms: k-nearest neighbours (k-NN), support vector machines (SVM), and Random Forests (RF). Particular attention is given to k-NN, an instance-based lazy-learning method that predicts the class of a new observation by identifying similar examples in its stored training data. The k-NN experiments involve several consequential design choices, including the distance measure, neighborhood size, voting strategy, and the policy governing the retention of training instances. These different choices may influence the classification's accuracy and efficiency, and their effectiveness may vary across datasets.

The experimental evaluation uses two datasets, namely the Heart-Statlog and TAO-Grid. Heart-Statlog represents a higher dimensional dataset containing both numerical and nominal features, but with less number of training examples. The TAO-Grid on the other hand is a low-dimensional dataset with only numerical features, but has more training examples. Both datasets are assessed using ten-fold cross validation partitions.

Additionally, this study considers the consistency of performance across folds, prediction execution time, and the number of training instances retained. These factors are relevant to k-NN because predictions depend directly on the stored case base. We also apply instance reduction techniques on the classifiers and investigate the retention patterns and impact on model performance.

The objectives of this study are to: (1) compare the classification of k-NN, SVM, and RF across datasets with different characteristics; (2) analyze how distance measures, neighborhood sizes, voting strategies, and retention policies influence k-NN classification; (3) analyze eager learning methods like SVM and RF across combinations of their respective hyperparameters; (4) assess the effects of instance reduction on predictive performance, time, and requirements for storage; and (4) examine the trade-offs between classification performance, stability, and efficiency across different experimental settings.

The remainder of the report describes the datasets and their feature representations, explains the experimental methodology and classifier configurations, presents the results of the parameter comparisons, and discusses the strongest observed configurations and the implications of the findings.

2 Data Preprocessing and Experimental Methodology

To give a better understanding of the data being evaluated, the table 1 below illustrates the characteristics of each dataset.

Heart-Statlog is a small dataset which contains 270 instances and 13 predictor features. Its attributes consist of 5 numeric and 8 nominal features. The target variable represents two classes: Absent at 150 instances or 55.56% of the dataset, and Present, at 120 instances or 44.44% of the dataset. Considering this, it is a

higher-dimensional classification problem with mixed feature types. There is class imbalance but it is not significant.

Attribute	Heart-Statlog	TAO-Grid
Instances	270	1,888
Target column	class	class
Predictor Features	13	2
Class Distribution	Absent 150 (55.56%) Present 120 (44.44%)	Black 944 (50.00%) White 944 (50.00%)
Numeric Features	5	2
Nominal Features	8	0

Table 1. Data characteristics of Heart-Statlog and TAO-Grid

The TAO-Grid on the other hand is a medium sized dataset that contains 1,888 instances and 2 continuous predictor features. Its target variable consists of two equal classes, Black and White, each containing 944 instances or 50% of the dataset.

2.1 Data Preprocessing and Leak Control

To prevent data leak, each dataset was evaluated using its predefined 10-fold cross-validation partitions. The folds are divided and arranged in order to keep similar class proportions in each training and test partition. The same folds are used for all configurations, making the comparisons of model performance fair.

For Heart-Statlog, each fold contains 243 training instances and 27 test instances. For the training partition, there are 135 Absent and 108 Present instances; the test partition has 15 Absent and 12 Present instances. This is in line with the original class distribution of 55.56% Absent and 44.44% Present.

For TAO-Grid, the training partition contains ≈ 1698 to 1700 training instances, while the test partition has ≈ 188 to 190 instances.

Dataset	Training Instances	Test Instances
Grid	≈ 1699	≈189
Heart	243	27

Figure 2 The ratios of training instances and test instances in each fold follow the original class distribution of each data set.

In order to prevent data leakage, all preprocessing methods are fitted only to the training partition of each fold. The results of these are then applied to both training and test partitions. This is to ensure that information from the test data does not influence preprocessing or model training. Training and test partitions were kept separate to prevent test data from influencing preprocessing or model fitting, which would make the evaluation overly optimistic.

2.2 Feature Preprocessing

The attribute data type information for the .arff files for heart-statlog was not reflective of the original nature of the data, as nominal features with numbers were labeled as numeric. Thus we defined a custom preprocessor to identify the same.

Feature-Type Identification. Feature types are inferred from the training partition, unless explicitly overridden. Boolean attributes, binary indicators encoded as 0/1, and integer-valued attributes satisfying the low-cardinality criteria are treated as nominal. Attributes containing non-numeric

string values are also classified as nominal, whereas attributes containing fractional values and integer-valued attributes not meeting the nominal criteria are treated as numerical. Integer-valued attributes are classified as nominal when the training partition contains at least 20 non-missing observations, at most 6 distinct values, a value range no greater than 10, and a ratio of distinct values to observations below 5%. Attributes containing only missing values default to numerical.

Missing-Value Imputation. Missing values are separated according to their feature type. In cases of missing numeric values, the median of the respective training feature will be assigned. For nominal values, they are replaced with the most frequent value (mode) calculated from the training partition.

Feature Normalization. For features that are continuous, they are normalized using min-max scaling. The minimum and maximum values exclusively from the training partition are used to transform both test and training partitions:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Constant features are assigned a value of zero

Categorical Encoding. Categorical features are converted into numeric values (one-hot encoded) so that algorithms such as k-NN, SVM, and RF are able to process these data. These will be useful in calculating distances for k-NN, decision boundaries for SVM, and representation for splits in RF. For nominal features, these are handled by HEOM and IVDM directly without one-hot encoding.

2.4 Instance Reduction

Instance reduction was implemented at the preprocessing stage so that investigation can be done using a smaller reference set without significantly reducing performance of predictions. The following reduction methods are tested: (1) Fast Condensed Nearest Neighbor 3 (FCNN3); (2) Second-Order Relative Neighborhood Graph (RNG2); and (3) DROP3. Reduced instances are then used as a reference set for the classification experiments.

This section will be discussed in detail in Section 3.4. It is important to note that all feature preprocessing is only applied to the training partition to avoid test data influencing preprocessing of training data.

3. K-Nearest Neighbor

The k-NN is a lazy learner, meaning that training only stores the instances and generalization is done only during query time, when the k closest store cases vote on the class (Aha et al., 1991). The assumption is that similar instances have similar classes. This makes it a good foundation and it is also the model that instance reduction acts on directly because the stored case base is the model.

3.1 Distance Measures

Six metrics are implemented: Euclidean, Manhattan, Canberra, Clark, Heterogeneous Euclidean-Overlap Metric (HEOM), and Interpolated Value Difference Metric (IVDM).

For numerical vectors (x, y) with m attributes, these distance measure are:

Euclidean (Mikowski p=2)

$$d(x,y) = \sqrt{\sum_{j=1}^{m}(x_j - y_j)^2}$$

Manhattan Distance

$$d(x,y) = \sum_{j=1}^{m}|x_j - y_j|$$

Canberra Distance

$$d(x,y) = \sum_{j=1}^{m}\frac{|x_j - y_j|}{|x_j|+|y_j|}$$

Clark

$$d(x,y) = \sqrt{\sum_{j=1}^{m}\left(\frac{|x_j - y_j|}{x_j + y_j}\right)^2}$$

Euclidean and Manhattan distances are used for standard benchmarking as they are widely used common distance metrics.

Canberra and Clark distance tests whether KNN benefits from emphasising proportional differences, particularly when numerical values vary considerably in magnitude. Although, coordinates near zero can contribute disproportionately.

HEOM combines numerical differences with categorical overlap:

HEOM

$$d_{HEOM}(x,y) = \sqrt{\sum_{j=1}^{m}\delta_j(x_j, x_j)^2}$$

For a nominal attribute, the attribute-level difference is zero when the values match and one otherwise. For a numerical attribute, it is the absolute difference divided by the attribute range. This allows categorical values to be compared without imposing an artificial numerical ordering.

Additionally, IVDM compares the conditional class-probability distributions associated with attribute values:

IVDM $[Euclidean]$

$$d(x,y) = \sqrt{\sum_{j=1}^{m}\sum_{c\epsilon C}(P(c|x_j) - P(c|y_j))^2}$$

Probabilities use Laplace smoothing. Numerical attributes are mapped to quantile bins, and each bin has an estimated class distribution. These statistics are fitted on the training case base only.

The inclusion of these heterogeneous distance functions is motivated by research on instance-based learning for mixed-type data (Wilson and Martinez, 1997).

Preprocessing is fitted separately within each training fold. Numerical attributes are normalized, while categorical attributes are encoded for the conventional numerical distances and preserved for the heterogeneous measures.

3.2 Neighbor Search and Voting

For each query, the classifier computes its distance to every stored instance and selects the k nearest cases using a stable sort. Equal distances preserve the current case-base order, making neighbor selection deterministic for a fixed input order. If fewer than k cases remain, all available cases are used.

Two voting strategies were implemented.

Majority voting gives each selected neighbor one vote:

$$\hat{y} = arg\,max_{c\epsilon C}\sum_{i\epsilon N_k(x)}1(y_i = c)$$

Inverse-distance voting gives closer neighbors more influence:

$$\hat{y} = arg\,max_{c \epsilon C} \sum_{i \epsilon N_k(x)} \frac{1(y_i = c)}{d(x, x_i)}$$

using a distance exponent of one. When zero-distance neighbors are present, only those neighbors participate in the vote, avoiding division by zero.

Voting ties are resolved by the closest individual neighbor belonging to a tied class, then by the training class prior, and finally by the lexicographic ordering of class labels.

Majority voting provides a simple reference strategy. Inverse-distance voting tests whether preserving the influence of the closest cases improves predictions when the neighborhood also contains less relevant examples.

3.3 Case Base and Retention Policies

The case base initially contains the training-fold instances, or the selected subset when instance reduction is applied. Four policies determine how it changes during evaluation:

Never Retain (NR)	Keeps the case base fixed.
Always Retain (AR)	Adds the query with its true label.
Different Class (DC)	Adds the query only when its prediction was incorrect.
Oblivion by Goodness (OG)	Updates retrieved cases' goodness and removes cases below the threshold.

Goodness starts at 0.5 and is updated as $G_t(c) = G_{t-1}(c) + 0.2[r(c) - G_{t-1}(c)]$, where $r(c) = 1$ if the retrieved case's label matches the query's true label and zero otherwise. Cases with updated goodness below 0.5 are removed. The default configuration does not retain newly solved queries.

These policies represent different accuracy-storage trade-offs. NR supplies a fixed-model reference; AR maximizes the incorporation of newly labeled cases; DC selectively retains mistakes; and OG tests whether removing unhelpful cases produces a smaller useful case base.

Evaluation is sequential: each query is predicted before its true label is used to update the case base. Thus, AR, DC, and OG describe an online setting with label feedback after each prediction. Their results must be interpreted in that setting, since later predictions can depend on earlier test instances. NR provides the conventional fixed-case-base comparison.

If OG empties the case base, the implementation predicts using the stored training class prior. Such predictions no longer use nearest-neighbor retrieval.

3.4 Instance Reduction Methods

Instance reduction methods aim to make the training set smaller, by selecting which data to keep or remove, in order to improve the model's ability to classify new data points or guess test fold data. This is beneficial because a smaller set can reduce storage and make predictions faster, which is beneficial particularly for k-NN as it compares a query with stored training data.

FCNN3

FCNN3 is a condensation method designed to produce a smaller reference set that

preserves the classification of the original training instances under the 1-nearest-neighbour (1-NN) rule. Following Angiulli (2007), the implementation initializes the subset with a representative of the most populated class.

At each iteration, it identifies the retained instance whose current Voronoi cell contains the largest number of opposing-class training instances, referred to as Voronoi enemies. It then adds the enemy closest to that retained instance. This process focuses selection on regions where the current subset fails to separate the classes correctly.

RNG2 RNG2 is an editing method based on the Relative Neighbourhood Graph (RNG) (Sánchez et al., 1997). It identifies potentially inconsistent instances through two successive tests.

First, an instance must be misclassified by the vote of its immediate RNG neighbours. Second, a strict majority of the expanded graph neighbourhood, formed from the instance and its same-class RNG neighbours, must belong to opposing classes. Only instances satisfying both conditions are removed.

DROP3 is a hybrid method of reduction that combines editing with condensation. It first applies an edited-nearest-neighbor-style filtering stage. It then considers removing surviving instances according to whether their removal reduces correct classification of their associates. Associates are instances whose neighbor sets include the case being considered for removal (Wilson & Martinez, 2000).

This method was selected to test whether combining noise filtering with removal of redundant cases offers a useful compromise between predictive performance and storage.

All three methods were applied to the training fold only, using Euclidean reduction distance. The subsequent k-NN prediction distance was varied independently. Where preprocessing differs between prediction distances, the representation supplied to the reducer may also differ; “Euclidean reduction” therefore does not necessarily imply an identical selected subset across every prediction-distance configuration.

3.5 Experiment Design

The experiments compared k-NN, SVM, and Random Forest on Heart-Statlog and TAO-Grid using the same ten predefined stratified folds. In each iteration, nine folds provided the training data and the remaining fold provided the test data. Preprocessing was fitted exclusively on the training partition and then applied to the test partition.

Each model was evaluated across the configuration grid specified in its implementation section. Two complementary comparisons were conducted: matched-setting comparisons, which held classifier parameters fixed to examine individual experimental factors, and best-configuration comparisons, which identified the strongest observed configuration within each tested grid.

Instance-reduction experiments compared the full training set with subsets produced by FCNN3, RNG2, and DROP3 using Euclidean reduction distance. Reduction was applied after preprocessing and

exclusively to training examples. Controlled k-NN reduction comparisons used no retention (NR), keeping the reduced case base fixed during testing. Adaptive retention policies were examined separately because they update the case base using labels revealed after each prediction. SVM and Random Forest remained fixed throughout test evaluation.

Predictive performance was summarized using pooled accuracy, mean fold accuracy with its standard deviation, balanced accuracy, and macro-F1. Computational comparisons considered fitting time, prediction time, and the proportion of training instances retained. Timing interpretations accounted for differences in measurement: k-NN processed queries sequentially, whereas SVM and Random Forest predicted complete test folds in batches. Training-instance counts were treated as measures of subset size rather than physical model memory.

Configuration selection and reporting used the same cross-validation results. Consequently, the selected maxima represent experimental tuning outcomes rather than independent estimates of final performance, and numerical differences were interpreted descriptively.

3.6 Results

The experiments evaluated six prediction distances, eight values of k ∈ {3, 5, 7, 9, 11, 13, 15}, two voting strategies, and four retention policies. This gives 384 configurations per dataset and reduction condition, each evaluated over ten folds. The conditions were unreduced k-NN, FCNN3, RNG2, and DROP3.

Results below use fold means. Where shown, "±" denotes the sample standard deviation across the ten folds. Accuracy is expressed as a percentage, while macro-F1 remains on a 0-1 scale. The comparisons are descriptive; numerical differences alone are not treated as evidence of statistical significance.

3.6.1 Distance Metric Results

Two complementary comparisons are presented. Table 1 compares distances at the same reference settings; Tables 2 and 3 show the best observed configuration for each distance after tuning.

Controlled Comparison at fixed settings. To compare distances while holding the remaining settings fixed, Table 1 uses unreduced k-NN with k = 3, majority voting, and NR.

Table 1. Accuracy by distance at fixed reference setting

Distance	Heart-Statlog Accuracy (%)	TAO-Grid Accuracy (%)
Euclidean	$82,22 \pm 6,49$	$95,82 \pm 1,62$
Manhattan	$81,48 \pm 4,62$	$95,71 \pm 1,62$
Canberra	$79,26 \pm 5,84$	$95,55 \pm 1,38$
Clark	$81,11 \pm 6,40$	$95,55 \pm 1,38$
HEOM	$82,22 \pm 6,49$	$95,82 \pm 1,62$
IVDM	$82,22 \pm 7,37$	$88,08 \pm 2,09$

On Heart-Statlog, Euclidean, HEOM, and the IVDM variant achieved the same mean accuracy at these settings, although IVDM showed greater fold variability. Canberra produced the lowest mean accuracy.

On TAO-Grid, Euclidean and HEOM again matched, while Manhattan, Canberra, and

Clark were close behind. The IVDM variant performed substantially worse, by approximately 7.74 percentage points relative to Euclidean. Mapping numerical values to shared bin-level class distributions may discard useful geometric distinctions, but this is a possible explanation rather than an experimentally isolated cause.

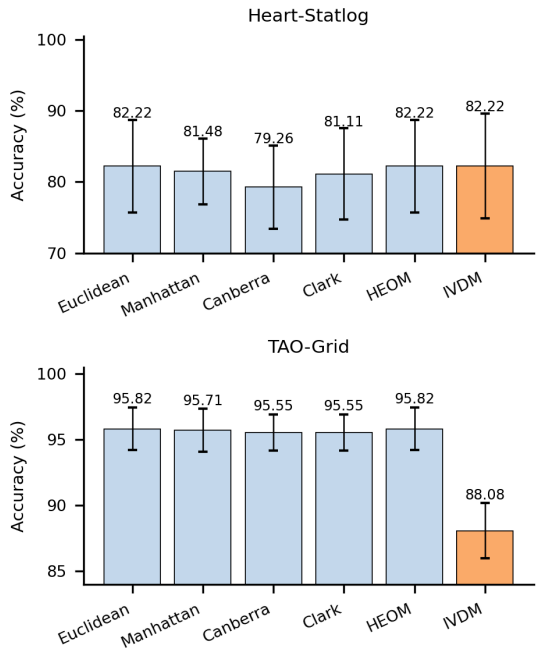


Figure 1. k-NN accuracy by distance metric on Heart-Statlog and TAO-Grid (k = 3, majority voting, no retention).

The k = 3 reference is not claimed to be optimal. Keeping k, voting, and retention fixed makes the rows directly comparable under one common setting. It does not describe each distance's highest achievable score within the tested Grid.

Best observed configuration per distance

For each distance and dataset, mean accuracy was calculated across the ten folds for every combination of eight k values, two voting strategies, and four retention policies. The configuration with the highest mean was selected. Thus, each row represents one configuration evaluated across all folds, rather than a different winner selected within each fold. All ties in mean accuracy are retained.

Table 2. Heart-Statlog: best observed unreduced k-NN configuration per distance

Distance	*k*	*Voting strategy*	*Retention policy*	*Mean accuracy (%)*
Euclidean	15	I-D	OG	84,81
Manhattan	9	I-D	NR / AR / DC	84,81
Canberra	7	M	AR	84,07
Canberra	15	I-D	NR	84,07
Clark	15	I-D	DC	84,07
HEOM	5	M	OG	84,44
HEOM	7	I-D	OG	84,44
IVDM variant	11	M	NR / DC	84,81

Note: I-D = Inverse-distance, M=Majority

Slash-separated policies indicate separate configurations tied in mean accuracy. Multiple rows for a distance indicate ties with different k values or voting strategies.

On Heart-Statlog, the highest mean accuracy was 84.81%, shared by Euclidean, Manhattan, and the IVDM variant. Euclidean reached this value at k = 15 with inverse-distance voting and OG; Manhattan at k = 9 with inverse-distance voting and NR, AR, or DC; and the IVDM variant at k = 11 with majority voting and NR or DC. Consequently, there is no unique distance winner by mean accuracy alone.

The tuned rankings also differ from the fixed-setting rankings. At the k = 3

reference, Manhattan obtained 81.48%, compared with 82.22% for Euclidean; after tuning, both reached 84.81%. This illustrates an interaction between distance and the other settings, rather than a contradiction between the tables. Manhattan's tied NR configuration additionally achieves that score with a fixed case base.

Table 3. TAO-Grid: best observed unreduced k-NN configuration per distance

Distance	*k*	*Voting strategy*	*Retention policy*	*Mean accuracy (%)*
Euclidean	13	I-D	AR	98,09
Manhattan	13	I-D	AR	98,25
Canberra	11	I-D	AR	97,83
Clark	15	I-D	AR	97.77
HEOM	13	I-D	AR	98.09
IVDM variant	9	I-D	NR / AR / DC	88.13
IVDM variant	9	M	NR / AR / DC	88.13

Note: I-D = Inverse-distance, M=Majority

For the IVDM variant, both voting strategies tie under each of NR, AR, and DC, giving six tied configurations.

On TAO-Grid, Manhattan achieved the highest mean accuracy, **98.25%**, at k = 13 with inverse-distance voting and AR. Euclidean and HEOM followed at 98.09% under the same settings. The difference from Manhattan was approximately 0.16 percentage points and is reported descriptively, without a claim of statistical significance.

The IVDM variant reached 88.13% at k = 9, so tuning did not close its gap with the other distances. Its quantile-bin representation may discard useful numerical distinctions, but this remains a possible explanation rather than an experimentally isolated cause.

3.6.2 Voting Strategy

Table 4 isolates voting using unreduced Euclidean k-NN with k = 3 and NR.

Table 4. Effect of voting strategy at fixed reference settings

Strategy	**Heart-Statlog Accuracy (%)**	**TAO-Grid Accuracy (%)**
Majority	$82,22 \pm 6,49$	$95,82 \pm 1,62$
Inverse Distance	$81,11 \pm 6,64$	$96,24 \pm 1,68$

Inverse-distance voting reduced Heart-Statlog accuracy by 1.11 percentage points at this configuration, but increased TAO-Grid accuracy by approximately 0.42 points. Weighting therefore did not provide a consistent improvement across datasets.

Nevertheless, inverse-distance voting appeared among the highest-accuracy configurations on both datasets. Its benefit depends on its interaction with the distance measure and neighborhood size, rather than on a universal advantage over majority voting.

3.6.3 Retention Policies

Table 5 compares Instance retention policies using unreduced Euclidean k-NN with k = 3 and majority voting. The size ratio is the final case-base size divided by its size before sequential evaluation.

Table 5. Accuracy and case-base change under retention policies

	NR	AR	DC	OG
Heart Accuracy (%)	82,22	81,85	81,48	82,96
Heart size ratio	1,000	1,111	1,021	0,927
Grid Accuracy	95,82	95,87	95,76	96,03
Grid size ratio	000	1,111	1,005	0,985

3.6.4 Instance Reduction Results

Table 6. Heart-Statlog instance reduction methods

Heart Statlog			
Method	**Accuracy (%)**	**Macro F1**	**Storage Reduction (%)**
Unreduced	$82,22 \pm 6,49$	0,8170	0,00
FCNN3	$80,74 \pm 4,55$	0,8029	64,90
RNG2	$82,59 \pm 7,62$	0,8192	9,96
DROP3	$81,85 \pm 4,08$	0,8133	83,29
Method	**$\bar{X}$ query time (ms)**		**Reduction time (s)**
Unreduced	0,0140		-
FCNN3	0,0082		0,0078
RNG2	0,0173		0,0436
DROP3	0,0061		0,0558

Table 7. TAO-Grid instance reduction methods

TAO Grid			
Method	**Accuracy (%)**	**Macro F1**	**Storage Reduction (%)**
Unreduced	$95,82 \pm 1,62$	0,9581	0,00
FCNN3	$95,23 \pm 1,44$	0,9523	92,20
RNG2	$95,82 \pm 1,62$	0,9581	0,18
DROP3	$96,40 \pm 1,65$	0,9640	89,72
Method	**$\bar{X}$ query time (ms)**		**Reduction time (s)**
Unreduced	0,0914		-
FCNN3	0,0083		0,0700
RNG2	0,0912		2,6554
DROP3	0,0104		10,2280

FCNN3 compressed both datasets significantly. On Heart-Statlog, it removed approximately 64.90% of the training cases with a 1.48-point decrease in accuracy. On TAO-Grid, it removed 92.20% with a decrease of approximately 0.58 points. Recorded query time was approximately 1.70 times faster on Heart-Statlog and 11.07 times faster on TAO-Grid.

RNG2 also removed training cases. It removed approximately 9.96% of Heart-Statlog's training cases and slightly increased mean accuracy. On TAO-Grid, it removed only 0.18%, leaving the reported accuracy and macro-F1 unchanged at the displayed precision. Its limited compression explains why it offered little prediction-time benefit. On Heart-Statlog, the measured query time was actually higher than the unreduced reference, showing that modest size reductions do not guarantee faster execution in separately recorded runs.

DROP3 achieved the largest compression on Heart-Statlog, removing 83.29% of cases while reducing accuracy by approximately 0.37 points. On TAO-Grid, it removed 89.72% and increased mean accuracy by approximately 0.58 points. Its recorded query times were approximately 2.30 and 8.76 times faster than the respective unreduced references.

These prediction-time improvements came with an initial reduction cost. In particular, DROP3 required approximately 10.23 seconds per TAO-Grid fold at the reference settings, compared with 0.07 seconds for FCNN3. Consequently, a smaller case base and faster queries do not automatically imply lower total runtime: the benefit depends on how many subsequent queries amortize the reduction cost. Figure 2 illustrates the storage reduction across datasets

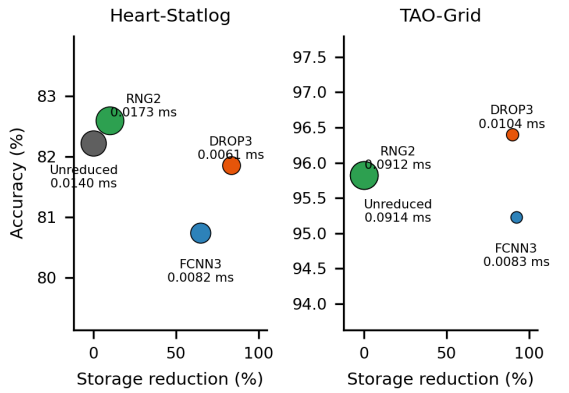


Figure 2: Plot of storage reduction across datasets

We plot the Heart statlog dataset by reducing its dimensionality to two features using Principal Component Analysis. Figure 3 illustrates the reduction and possible classification boundary on a 2D plot. No dimensionality reduction is needed for TAO_grid as it has only 2 features.

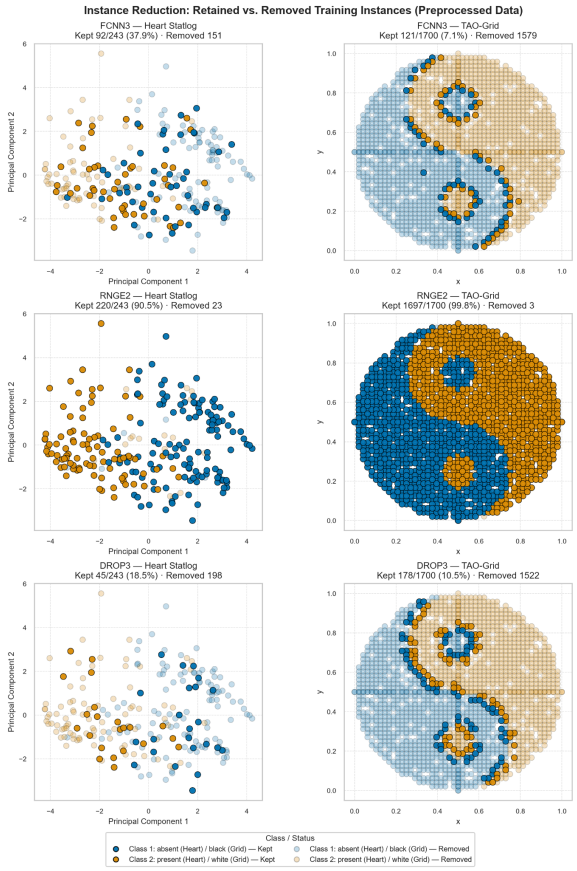


Figure 3: Instance reductions plotted on 2D plane.

4 Support Vector Machine

4.1 Definition

A Support Vector Machine (SVM) is a supervised classification method that learns a decision boundary by balancing a wide separation margin with penalties for margin violations. This soft-margin formulation allows classification of data that are not perfectly separable (Cortes & Vapnik, 1995). The training examples that determine the fitted boundary are called support vectors.

The regularization parameter C controls this balance: larger values penalize margin violations more strongly, while smaller values permit more violations in exchange for stronger regularization.

Two kernels were evaluated in this project. The linear kernel produces a linear decision boundary in the processed feature space, while the Radial Basis Function (RBF) kernel allows

4.2 Implementation

The SVM experiments evaluated linear and RBF kernels with C fixed at 1.0 and gamma set to “scale” for RBF. Both kernels were tested without reduction and with FCNN3, RNG2, and DROP3, using Euclidean distance for reduction and ten predefined folds per dataset. Table shows the SVM configuration and evaluation settings.

Kernels	**Linear** $K(x, z) = x^{\top} z,$ **Radial Basis Function (RBF)** $K(x, z) = exp - (\gamma \|x - z\|)^2$
Regularization Parameter	$C = 1.0$
RBF Coefficient	igugamma = “scale”
Reduction Conditions	None, FCNN3, RNG2, DROP3
Reduction Distance	Euclidean
Evaluation	10 predefined folds per data =set

Table 8. SVM configuration and evaluation settings.

The scale setting calculates the RBF coefficient from the feature count and variance of the matrix supplied to fitting. Consequently, its numerical value can change after instance reduction. It is ignored by the linear kernel.

The complete experiment contains **160** evaluations, derived from 4 reduction conditions × 2 kernels × 2 datasets × 10 folds. All configurations completed all ten folds.

The classifier remains fixed during test evaluation. Test labels are used to calculate metrics rather than update the fitted model. Fitting time is measured around the model-training call, while prediction time is measured for the complete test fold and divided by the number of examples. The resulting per-instance timing is therefore an amortized batch measurement. Instance-reduction time is not recorded separately and is excluded from the measured SVM fitting time.

4.3 Rationale

SVM was selected as a margin-based eager learner that constructs a decision function during training. It allows the study to examine classification through a learned boundary rather than relying exclusively on neighborhood retrieval.

The linear kernel provides a simple reference model, testing whether a linear boundary is sufficient in the processed feature space. On the other hand, RBF provides a more flexible alternative for investigating nonlinear class relationships.

For instance reduction techniques, applying FCNN3, RNG2, and DROP3 before SVM fitting examines whether subsets selected using neighborhood-based criteria remain useful for a margin-based classifier. These reduction methods do not directly optimize the SVM objective, so preserving useful nearest-neighbor structure does not guarantee that they preserve the examples needed to learn an effective SVM boundary.

The experiment holds C = 1.0 and the RBF coefficient rule fixed across conditions. It

therefore evaluates the effect of each reduction pipeline under common settings, rather than finding an independently optimized SVM for every reduced training set.

4.4 Results and Interpretations

Data set	Kernel	Mean Accuracy (%) ± SD (%)	Mean Balanced Accuracy (%)
Heart Statlog	Linear	84,81 ± 5,91	84,17
	RBF	81,85 ± 4,43	81,33
TAO Grid	Linear	83,68 ± 2,36	83,68
	RBF	91,42 ± 1,55	91,41

Table 9. Comparison between Linear and RBF Kernel for SVM.

SVM kernel performance depended on the dataset. On Heart-Statlog, linear SVM achieved 84.81% pooled accuracy, exceeding RBF's 81.85% by 2.96 percentage points. Although RBF showed lower fold-to-fold variability, its lower accuracy indicates that the nonlinear kernel offered no predictive advantage at the evaluated settings.

On TAO-Grid, RBF achieved 91.42% pooled accuracy, compared with 83.69% for linear SVM, an improvement of approximately 7.73 percentage points. This finding is consistent with a nonlinear decision boundary being useful for this dataset. Balanced accuracy and macro-F1 supported the accuracy rankings on both datasets.

Overall, linear SVM performed better on Heart-Statlog, whereas RBF performed better on TAO-Grid. These findings apply to the tested configurations and do not establish statistical significance or demonstrate that either kernel is universally preferable. As illustrated in figure 4.

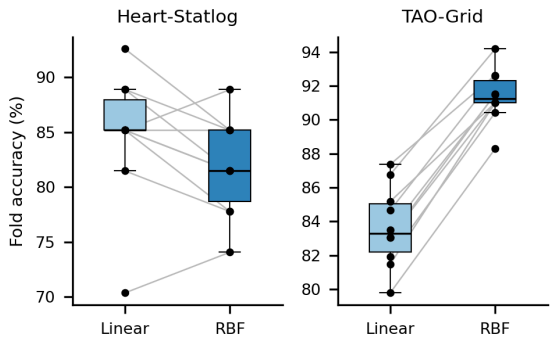


Figure 4. Fold-wise SVM Kernel Accuracy Comparison on Heart-Statlog and TAO-Grid

4.5 Effect of Instance Reduction on SVM Performance

The tables below compare accuracy, training-set reduction, and execution time against the unreduced reference for each kernel. Reduction percentages pool training-instance counts across folds. Fitting times are fold averages, while prediction times are total prediction times divided by total test examples.

Data set	Reduction	Linear Accuracy (%)	RBF Accuracy (%)	Instances removed (%)
Heart Statlog	None	**84,81**	81,85	0,00
	FCNN3	79,26	80,37	64,90
	RNG2	82,59	**83,70**	9,96
	DROP3	80,74	82,96	83,29
TAO Grid	None	83,69	**91,42**	0,00
	FCNN3	69,07	86,23	92,20
	RNG2	**83,79**	91,37	0,18
	DROP3	70,82	88,72	89,72

Table 10. Pooled SVM accuracy and percentage of training instances removed under each reduction method.

Reduction	Linear Fitting (ms/fold)	RBF Fitting (ms/fold)	Linear Prediction (%)	
None	**84,81**	81,85	0,00	
FCNN3	79,26	80,37	64,90	
RNG2	82,59	**83,70**	9,96	
DROP3	80,74	82,96	83,29	
TAO Grid				
None	83,69	**91,42**	0,00	
FCNN3	69,07	86,23	92,20	
RNG2	**83,79**	91,37	0,18	
DROP3	70,82	88,72	89,72	

RNG2 provided the highest accuracy among the reduced configurations for both kernels on both datasets. On Heart-Statlog, it removed 9.96% of training instances and increased RBF accuracy from 81.85% to 83.70%. On TAO-Grid, it removed only 0.18%, leaving accuracy and prediction time almost unchanged.

FCNN3 and DROP3 achieved substantially greater compression and reduced fitting and prediction times. However, FCNN3 lowered accuracy for both kernels on both datasets. DROP3 improved RBF accuracy on Heart-Statlog but reduced linear accuracy there and both kernels' accuracy on TAO-Grid. On TAO-Grid, DROP3 preserved more accuracy than FCNN3 despite removing fewer instances.

For RBF on TAO-Grid, FCNN3 and DROP3 provided prediction speedups of approximately 3.71× and 2.89×, accompanied by accuracy decreases of 5.19 and 2.70 percentage points, respectively. These results illustrate the trade-off between compression, prediction speed, and predictive performance.

The highest overall SVM accuracy remained with unreduced linear SVM on Heart-Statlog and unreduced RBF SVM on TAO-Grid. Reduction time was excluded from fitting measurements, so faster fitting and prediction do not establish lower total runtime.

5 Random Forest

Random Forest (RF) is an ensemble learning method that combines multiple decision trees. Each tree is trained on a bootstrap sample of the training data and considers a random subset of features at each split. This randomness encourages diversity among trees, helping reduce the variability associated with a single decision tree (Breiman, 2001).

In the implementation used in this project, classification is performed by averaging the trees' estimated class probabilities and selecting the class with the highest average probability. The ensemble can represent nonlinear relationships and interactions between features.

Two split criteria were evaluated: Gini impurity and entropy. Both measure class impurity within a node and guide the selection of splits that produce purer child nodes.

5.1 Implementation

The implementation uses scikit-learn's RandomForestClassifier through a wrapper that trains the forest, predicts test labels, and records performance metrics. A fresh

classifier is created for each configuration and fold. The implementation supports parallel execution across trees and uses a configurable random seed, defaulting to 42. The table below summarizes the RF configuration and evaluation settings.

Setting	Estimated Values
Number of Trees	100 and 200
Split Criterion	Gini and entropy
Maximum tree depth	Unrestricted
Reduction conditions	None, FCNN3, RNG2, DROP3
Reduction Distance	Euclidean
Evaluation	Ten predefined folds per dataset

Table 10. Random Forest configuration and evaluation settings.

Following the shared experimental design, preprocessing is fitted exclusively on training data. When enabled, instance reduction is applied before forest fitting, while the fitted model remains fixed throughout test evaluation.

Fitting and prediction times are measured separately. Prediction time is recorded for the complete test fold and divided by its number of examples, giving an average batch prediction time per instance. Reduction time is excluded from the reported fitting time. Results are aggregated by dataset, configuration, and reduction condition.

5.2 Rationale

Random Forest was selected to compare an ensemble of decision trees with the neighborhood-based approach of k-NN and the margin-based approach of SVM. Its ability to represent nonlinear relationships and feature interactions provides a useful alternative for examining the two datasets.

Comparing forest sizes allows the experiment to assess whether additional trees improve predictive performance enough to justify their computational cost. Evaluating Gini and entropy also examines whether the choice of splitting criterion affects classification under otherwise common settings.

The reduction experiments investigate whether subsets selected using neighborhood-based methods remain effective for training a tree ensemble. Removing examples may reduce redundancy and fitting cost, but it can also remove information needed to construct useful splits. Applying the same folds and configuration grid across reduction conditions supports comparison of these effects on accuracy, training-set size, and execution time.

5.3 Results and Interpretation

Without instance reduction, the highest observed Random Forest accuracy on both datasets was obtained using 200 trees and Gini impurity.

On Heart-Statlog, this configuration achieved 81.85% pooled accuracy. Increasing the forest from 100 to 200 trees improved accuracy under both splitting criteria, by approximately 0.37 percentage points for Gini and 1.48 points for entropy. However, the 200-tree Gini configuration also showed the greatest fold-to-fold variability, indicating that its higher average performance was not accompanied by greater consistency.

Data set	Trees	Criterion	Mean Accuracy (%	Std. Dev. (%)	Mean Fit Time (s)	Mean Prediction Time (ms/query)
Heart Statlog	100	Gini	81.85	5.64	0.1992	1.1367
	100	Entropy	80.74	6.25	0.1849	1.0015
	200	Gini	82.22	6.49	0.3651	1.6985
	200	Entropy	81.85	6.16	0.3671	1.5583
TAO Grid	100	Gini	96.82	1.45	0.2388	0.2151
	100	Entropy	96.51	1.64	0.2418	0.1932
	200	Gini	96.88	1.46	0.4529	0.3050
	200	Entropy	96.72	1.72	0.4574	0.3204

Table 11. Random Forest accuracy, fitting time, and prediction time without instance reduction on Heart-Statlog and TAO-Grid.

On TAO-Grid, 200 trees with Gini achieved 96.88% pooled accuracy. Gini outperformed entropy at both forest sizes, while increasing from 100 to 200 Gini trees improved accuracy by less than 0.1 percentage points. Thus, doubling the ensemble size provided little additional predictive benefit on this dataset.

Overall, larger forests improved accuracy within the evaluated grid, but the gains depended on the dataset and splitting criterion. Whether these improvements justify additional computational cost should be assessed using the corresponding timing results. These comparisons are descriptive and do not establish statistical significance.

Overall, increasing the number of trees provided only small accuracy improvements while increasing computation time, as illustrated by figure 5. These results are descriptive and do not imply statistical significance.

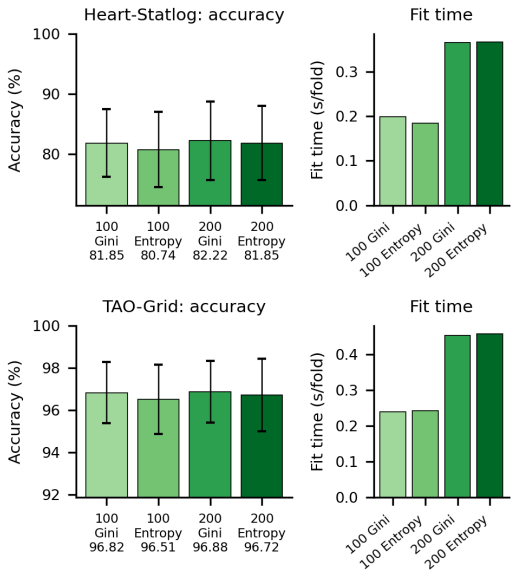


Figure 5. Random Forest Performance and Fit Time Comparison

5.3 Instance Reduction

Table 11 holds the forest configuration fixed at 200 trees Gini impurity. This separates the effect of the reduction pipeline from changes in forest size or split criterion. Training-instance reduction is calculated as

$$TIR\ (\%)\ =\ 100\left(1-\frac{\sum_{f}|S_f|}{\sum_{f}|T_f|}\right)$$

where Tf is the original training fold and Sf is the subset supplied to RF fitting.

Dataset	Reduction	Pooled Accuracy (%)	Change from baseline (pp)
Heart Statlog	None	81,85	-
	FCNN3	80,00	-1,85
	RNG2	83,70	+1,85
	DROP3	84,07	+2,22

Dataset	Reduction	Pooled Accuracy (%)	Change from baseline (pp)
TAO Grid	None	96,88	-
	FCNN3	95,60	-1,27
	RNG2	96,72	-0,16
	DROP3	94,54	-2.33

Table 12. Effect of instance reduction on Random Forest pooled accuracy using 200 trees and Gini impurity.

Dataset	Reduction	Mean macro-F1	Removed instances(%)
Heart Statlog	None	0,8136	0
	FCNN3	0,7951	64,90
	RNG2	0,8277	9,96
	DROP3	0,8358	83,29

Dataset	Reduction	Mean macro-F1	Removed instances(%)
TAO Grid	None	0,9688	0
	FCNN3	0,9560	92,20
	RNG2	0,9672	0,18
	DROP3	0,9454	89,72

Table 13. Random Forest mean macro-F1 and percentage of training instances removed using 200 trees and Gini impurity.

On Heart-Statlog, RNG2 and DROP3 improved accuracy at the fixed reference settings. RNG2 removed approximately 9.96% of training examples and increased accuracy by 1.85 percentage points. DROP3 removed 83.29%and increased accuracy by 2.22 points. Macro-F1 supported both improvements.

These outcomes are consistent with the selected subsets removing unhelpful examples or redundancy. However, they do not prove that the removed cases were noisy or identify the precise cause of the improvement. FCNN3 also produced substantial compression,but its accuracy decreased by approximately 1.85 points.

On TAO-Grid, every reducer decreased accuracy at the matched settings. FCNN3 removed 92.20% of training instances with a 1.27-point decrease, while DROP3 removed 89.72% with a 2.33-point decrease. RNG2 removed only 0.18%,producing a much smaller accuracy change.

The results demonstrate that the usefulness of reduction depends on the dataset and method. Strong compression improved the Heart-Statlog reference under DROP3 but reduced TAO-Grid performance.

8. Statistical Analysis Framework

The evaluation framework follows the non-parametric methodology outlined by Demšar (2006), deliberately avoiding cross-validation paired t-tests to prevent falsely elevated Type I error rates caused by fold overlap and data correlation. For each dataset individually, 10-fold cross-validation results are pooled across held-out test instances to calculate descriptive metrics: pooled classification accuracy, mean query efficiency in milliseconds, and instance retention ratios. In multi-dataset comparisons across independent benchmark domains (heart-statlog and electrical grid stability), configurations are ranked across datasets and assessed using the non-parametric Friedman omnibus test with exact tie corrections, alongside the Iman-Davenport F-statistic and post-hoc

critical difference analysis. Because the benchmark suite comprises two datasets (D = 2), multi-dataset inferential p-values are interpreted cautiously and grounded by dataset-specific leaderboards.

Across all evaluated models (k-Nearest Neighbors, Support Vector Machines, and Random Forests), lazy learning with distance-weighted voting proved exceptionally competitive in predictive accuracy. Specifically, k-NN configurations utilizing Manhattan and HEOM (Heterogeneous Euclidean-Overlap Metric) distance functions with inverse-distance weighting and moderate neighborhood sizes (k between 7 and 15) led the overall multi-dataset accuracy standings, achieving approximately 98.0% pooled accuracy on the grid dataset and 84.4% to 84.8% on heart-statlog.

In contrast, eager models demonstrated substantial advantages in inference latency, highlighting a clear operational trade-off. Linear Support Vector Machines achieved the fastest evaluation speed by a significant margin, requiring only 0.011 milliseconds per query on the grid dataset and 0.023 milliseconds on heart-statlog, vastly outperforming k-NN's computational demand during exhaustive neighbor searches. Finally, instance reduction and retention strategies successfully mitigated lazy learning's memory and computation bottlenecks: techniques such as Order-of-Growth retention reduced stored training instances down to approximately 68% to 70% on heart-statlog and 85% to 90% on grid, effectively accelerating query-time distance calculations while preserving strong decision boundaries.

9. Comparison

Comparing k-NN, SVM, and Random Forest showed that their relative performance depended on the dataset and the training examples available. k-NN bases its predictions on nearby stored cases, SVM learns a decision boundary controlled by its kernel and regularization, and Random Forest combines the predictions of multiple decision trees. These differences influenced both predictive performance and the effect of instance reduction.

For the unreduced experiments, linear SVM and the best k-NN configuration with no retention achieved the same accuracy of 84.81% on Heart-Statlog, exceeding the best Random Forest result of 81.85%. On TAO-Grid, k-NN with no retention achieved 97.93%, followed by Random Forest at 96.88% and RBF SVM at 91.42%. Thus, k-NN performed competitively on both datasets, while Random Forest was particularly effective on TAO-Grid. SVM depended strongly on kernel selection: linear performed better on Heart-Statlog, whereas RBF performed better on TAO-Grid.

Instance reduction further distinguished the three classifiers. Because k-NN searches its stored examples during prediction, reducing the case base can directly lower its search cost. SVM and Random Forest instead predict using their fitted models, so removing training examples does not necessarily produce comparable prediction-time savings. The experiments reflected this distinction: substantial compression produced clear query-time improvements in the earlier k-NN evaluation, faster fitting and prediction for several SVM configurations, and smaller timing changes for Random Forest.

Accuracy after reduction also depended on the classifier. At the matched reference settings, DROP3 improved k-NN accuracy on TAO-Grid but decreased SVM and Random Forest accuracy. Conversely, RNG2 and DROP3 improved some Heart-Statlog SVM and Random Forest results. Therefore, preserving useful neighborhood structure does not guarantee that a reduced subset will preserve an effective margin-based boundary or tree ensemble.

Together, these findings support selecting the classifier and reduction method jointly. k-NN offered strong predictive performance and a direct opportunity to reduce case-base search costs, SVM benefited from dataset-appropriate kernel selection, and Random Forest provided strong TAO-Grid performance with comparatively modest timing benefits from reduction. These conclusions apply to the tested configurations; differences in evaluation procedures and configuration selection limit claims of general superiority.

10. Conclusion

The experiments demonstrated that the performance of k-NN, SVM, and Random Forest depended on the dataset, configuration, and instance-reduction method. These models represent different learning approaches: k-NN classifies through stored examples, SVM learns a margin-based decision boundary, and Random Forest combines multiple decision trees. Their results show that model complexity alone does not determine predictive performance.

Without instance reduction, linear SVM and the best k-NN configuration using no retention both achieved 84.81% accuracy on Heart-Statlog, compared with 81.85% for the best Random Forest configuration. On TAO-Grid, k-NN with no retention achieved 97.93%, followed by Random Forest at 96.88% and RBF SVM at 91.42%. Linear SVM performed better than RBF on Heart-Statlog, whereas RBF performed better on TAO-Grid, demonstrating the importance of selecting a suitable kernel. Adaptive k-NN configurations achieved higher TAO-Grid accuracy, but their use of sequential label feedback distinguishes them from the fixed-model evaluation.

Instance reduction introduced a trade-off between training-set size, predictive performance, and computational cost. FCNN3 and DROP3 produced substantial compression, while RNG2 generally retained more examples. However, their effects differed across classifiers. In the earlier k-NN reduction experiments, DROP3 improved TAO-Grid accuracy while substantially reducing the case base; the corresponding SVM and matched Random Forest experiments showed accuracy decreases. This indicates that a subset selected using neighborhood-based criteria does not necessarily preserve the information required by other learning algorithms.

Overall, instance reduction was particularly relevant to k-NN because prediction directly depends on the stored case base. For SVM and Random Forest, fewer training examples did not guarantee proportional prediction-time savings. The choice of model and reducer should therefore reflect the required balance between accuracy, storage, and execution time. These findings remain descriptive: the study used two

datasets, configuration selection relied on the reported folds, and timing procedures differed between model families.

11 References

Aha, D. W., Kibler, D., & Albert, M. K. (1991). Instance-based learning algorithms. *Machine Learning, 6*(1), 37–66. https://doi.org/10.1007/BF00153759

Angiulli, F. (2007). Fast nearest neighbor condensation for large data sets classification. *IEEE Transactions on Knowledge and Data Engineering, 19*(11), 1450–1464. https://doi.org/10.1109/TKDE.2007.190645

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Sánchez, J. S., Pla, F., & Ferri, F. J. (1997). Prototype selection for the nearest neighbour rule through proximity graphs. *Pattern Recognition Letters, 18*(6), 507–513. https://doi.org/10.1016/S0167-8655(97)00035-4

Wilson, D. R., & Martinez, T. R. (1997). Improved heterogeneous distance functions. *Journal of Artificial Intelligence Research, 6*, 1–34. https://doi.org/10.1613/jair.346

Wilson, D. R., & Martinez, T. R. (2000). Reduction techniques for instance-based learning algorithms. *Machine Learning, 38*(3), 257–286. https://doi.org/10.1023/A:1007626913372